Broomstick Management System

Name: Radhey Sharma
UFID: 21089036
Mail: radheysharma@ufl.edu

Instructor: Dr. Sartaj K Sahni
COP5536 Advanced Data Structures
Department of CISE
University of Florida

Table of Contents

Introduction 1
Project Details 2
Program Structure 3
rbt.h 3
Private members: 3
Public members: 5
rbt.cpp 6
main.cpp 6
Helper Functions: 6

Introduction

The aim of this project is to design a new flying broomstick management system for Office of Transportation, Ministry of Magic. The system, aiming at managing thousands of manned cleaning tools manufactured and registered every year. According to laws and regulations, all flying broomsticks must bear a license plate with 4 characters. Each character can either be a number or a capital letter. When registering a flying broomstick, the owner can either customize their plate by choosing their own plate number or let the system randomly generate one. Registration and management fee for a plate is 4 Galleons annually, with an extra 3 Galleons charged for customized plate number per year.

Project Details

The new flying broomstick management system will be designed with the help of Red-Black Tree as underlying data structure and dictionary-style data entries with the license plate numbers as key. This will be programmed in the C++ language and will make use of the object-oriented features like classes to define a Red-Black tree and its operations. The user will submit the operations to be performed in a text file and the results will be saved in another text file.

The three major operations that can be performed in a Red-Black Tree are **Search**, **Insertion** and **Deletion**. These operations will be performed with the help of several helper functions. These operations will be used to perform the operations defined for the management system which are:

- **addLicence(plateNum):** register new customized license plate into the database.
- **addLicence():** randomly create a new license plate that does not currently exist and add into the database.
- **dropLicence(plateNum):** remove record of a license plate from the database.
- **lookupLicence(plateNum):** check in the database if the license plate exists in it.
- **lookupPrev(plateNum):** check in the database for the license plate number lexicographically previous to the input.
- **lookupNext(plateNum):** check in the database for the license plate number lexicographically next to the input.
- **lookupRange(lo, hi):** check in the database and output all license plates between lo and hi in lexicographical order.
- **revenue():** report the annual revenue on broom registration and management fee and customized plate number fee.
- **quit():** stop the program.

Program Structure

The program consists of a makefile and three program files:

- rbt.h
- rbt.cpp
- main.cpp

rbt.h

This file defines the Red-Black Tree class RBT, its member variables and function prototypes as well as the node structure of the tree. It also contains the libraries required to run the program.

The libraries are:

- **iostream:** To handle standard I/O operations.
- **fstream:** To handle the file reading and writing operations.
- **cstdlib:** To do random number generation required for the management system operations.
- **string:** To deal with string data type.

The Node structure is used to store data related to the Red-Black Tree data structure with the plate number as its key.

- **string plateNum:** Stores the plate number of a licence.
- **bool red:** Keeps track of colour of node in Red-Black Tree.
- **bool custom:** Keeps track of licence plate type. (whether its custom built or randomly generated)
- **Node *left:** Points to the left child of node.
- **Node *right:** Points to the right child of node.
- **Node *parent:** Points to the parent of node.
- **Node():** It is a constructor for the node to initialize a node with default values.

The class RBT is defines the Red-Black Tree member variables and functions:

Private members:

Variables:

- **Node *root:** Points to the root node of the tree.
- **bool found:** Checks whether a given licence plate is in the tree or not.
- **int amount:** Used to store the annual revenue of the management system.

Functions:

- **void leftRotate(Node *x)**
  Takes pointer to a node as an argument and performs the left rotation operation on the node.
- **void rightRotate(Node *y)**
  Takes pointer to a node as an argument and performs the right rotation operation on the node.
- **void setPlate(Node *node, string plate)**
  Takes pointer to a node and a string as an argument and stores it in the plateNum variable of the node.
- **int pltCompare(string a, string b)**
  Takes two strings as an argument and returns the lexicographical order of the strings: 1 means string a comes before string b; -1 means string b comes before string a; 0 means both strings are same.
- **Node* searchInsert(string plate, Node *node, bool insert)**
  Takes string and pointer to a node and a Boolean as its argument. This function searches if the plate exists in the system and returns pointer to the node that contains the plate number if the insert Boolean is set to 0, otherwise it inserts the plate number in the system and returns pointer to the newly created node.
- **void fixInsert(Node *node)**
  Takes a node pointer as its argument. This is a helper function that recursively resolves the Red-Black Tree violations that are created during insertion of the new node.
- **bool insert(string plate, bool cust)**
  Takes string and boolean as its argument. This function performs insertion by calling searchInsert function and returns whether it was successful or not. It also calls fixInsert to fix the violations and informs the system of custom insertions by saving the boolean in the custom variable of the node.
- **Node *predecessor(Node *node)**
  Takes node pointer as its argument and returns the pointer to its inorder predecessor node.
- **Node *successor(Node *node)**
  Takes node pointer as its argument and returns the pointer to its inorder successor node.
- **void transplant(Node* u, Node* v)**
  Takes two node pointers as its argument and transplants node v in place of node u. This is used in cases of deletion of node with one child.

- **void fixDelete(Node *node)**
  Takes a node pointer as its argument. This is a helper function that recursively resolves the Red-Black Tree violations that are created during deletion of a node.
- **bool deleteNode(string plate, Node *start)**
  Takes string and boolean as its argument. This function performs deletion of a node and returns whether it was successful or not. It also calls fixdelete to fix the violations.
- **char random()**
  This function returns a randomly generated alphanumeric character.
- **string randomPlate();**
  This function the random function multiple times to generate a random plate number with 4 characters that does not already exist in the management system.
- **string inRangeInorder(Node *node, string lo, string hi, bool first)**
  Takes a node pointer, two strings that define the range and a bool as its argument. This function is used to perform inorder traversal of a tree in a specified range. The boolean keeps track of the first node within range during traversal.

Public members:

Functions:

- **RBT()**
  This is the constructor for the RBT object. This initializes the private variables of the RBT class object and also sets the seed for pseudo random number generator rand() function with the srand() function with the current time as its input.

  The following functions are the operations defined in the project details. Here are their output formats:

- **string addLicence(string plateNum)**
  Output format:
  – If added successfully: %plateNum% registered successfully.
  – If %plateNum% already exists in database: Failed to register %plateNum%: already exists.
- **string addLicence()**
  Output format: %plateNum% created and registered successfully.

- **string dropLicence(string plateNum)**
  Output format:
  – If removed successfully: %plateNum% removed successfully.
  – If %plateNum% does not exist in database: Failed to remove %plateNum%: does not exist.
- **string lookupLicence(string plateNum)**
  Output format:
  – If exists: %plateNum% exists.
  – If not: %plateNum% does not exist.
- **string lookupPrev(string plateNum)**
  Output format:
  – If exists: %plateNum%’s prev is %plateNumFound%.
  – If not: %plateNum%’s prev does not exist.
- **string lookupNext(string plateNum)**
  Output format:
  – If exists: %plateNum%’s next is %plateNumFound%.
  – If not: %plateNum%’s next does not exist.
- **string lookupRange(string lo, string hi)**
  Output format: plate numbers between %lo% and %hi%: %plateNum%, %plateNum%, %plateNum%.
- **string revenue()**
  Output format: Current annual revenue is %amount% Galleons.

rbt.cpp

This file contains the logic of the function prototypes defined in the rbt.h file.

main.cpp

This file contains the main function along with other helper functions.

Helper Functions:

- **bool alphaNumeric(char c)**
  Takes a character as an input and returns whether it is an alphanumeric character or not.
- **string parser(string input)**
  Takes a string from the text file as its input and extracts the parameters from the defined operations, for example: it extracts "AAAA" from addLicence(AAAA).

- **string outputName(string input)**
  Takes string as its input which is the user input file name and if read correctly, returns the corresponding output file name. Otherwise returns “namingError”.

The main function creates a RBT object, and opens the specified input file. If syntax is correct and the file opens successfully, the program creates a corresponding output file, reads the input file queries line by line, performs the specified operations and then writes the result to the output file for each query. Otherwise, it will print an error message in the terminal.